

OpenFeign client implementation

OpenFeign client is another alternative that helps with executing the REST API calls. Its main advantage is that it removes the boilerplate code. Have a look at the code mentioned in step 3. It is sleek, readable, and contains less code compared to others.

It just needs a Java interface that has the REST API signatures—that's it. Then, you can use it. For each service, you can define a separate Feign interface. It works very well with Eureka. Use of OpenFeign requires the following steps:

1. First, you need to add a new dependency in the `pom.xml` file on `booking-service`, as shown in the following example:

```
<!-- OpenFeign client dependency -->
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-openfeign</artifactId>
</dependency>
```

2. Next, you need to add the `@EnableFeignClients` annotation in the main class of `booking-service` to mark that `booking-service` would use the OpenFeign client, as shown in the following example:

```
@SpringBootApplication
@EnableEurekaClient
@EnableFeignClients
public class BookingApp {
    ... omitted
}
```

3. Then, you need to define the REST resource interface annotated with `@FeignClient`. Because we want to consume `user-service` APIs, we'll assign it to the `FeignClient` annotation. Then, you define the signatures and request mapping of the APIs you want to consume. This is demonstrated in the following code:

```
@Component
@FeignClient("user-service")
public interface UserFeignClient {

    @RequestMapping(method = RequestMethod.GET, value = "/v1/user")
    Collection<UserVO> getUser(@RequestParam("name") String name) throws Exception;

    @RequestMapping(method = RequestMethod.POST, value = "/v1/user")
    UserVO postUser(@RequestBody UserVO user) throws Exception;

    @RequestMapping(method = RequestMethod.PUT, value = "/v1/user/{id}")
    void putUser(@PathVariable("id") long id, @RequestBody UserVO user) throws Exception;

    @RequestMapping(method = RequestMethod.DELETE, value = "/v1/user/{id}")
    void deleteUser(@PathVariable("id") long id) throws Exception;
}
```

Feign client is ready to consume `user-service`.

4. We add another command-line runner for demonstrating OpenFeign client usage, as shown in the following example:

```
@Component
@ConditionalOnProperty(prefix = "command.autorun", name = "enabled", havingValue = "true")
class OpenfeignClientSample implements CommandLineRunner {

    private static final Logger LOG = LoggerFactory.getLogger(OpenfeignClientSample.class);

    @Autowired
    private UserFeignClient userFeignClient;

    @Override
    public void run(String... strings) throws Exception {
        LOG.info("Creating new user");
        UserVO userVO = new UserVO();
        userVO.setId("5");
        userVO.setName("Y User");
        userVO.setAddress("Y Address");
        userVO.setCity("Y City");
        userVO.setPhoneNo("1234567890");
        try {
            UserVO newUser = userFeignClient.postUser(userVO);
            assert newUser.getId() == "5";
        } catch (Exception e) {
            LOG.error(e.getMessage());
        }
        LOG.info("\n\n\n\nUpdate newly created user");
        userVO = new UserVO();
        userVO.setId("5");
        userVO.setName("Y User 1");
        userVO.setAddress("Y Address 1");
        userVO.setCity("Y City 1");
        userVO.setPhoneNo("1234567890");
        try {
            userFeignClient.putUser(5, userVO);
        } catch (Exception e) {
            LOG.error(e.getMessage());
        }
        LOG.info("\n\n\nRetrieve users again\n\t\t\tto check if newly created object got updated");
        try {
            userFeignClient.getUser("y").forEach((UserVO user) -> {
                LOG.info("GET /v1/user --> {}", user);
            });
        } catch (Exception e) {
            LOG.error(e.getMessage());
        }
        LOG.info("\n\n\n\nDelete newly created user");
        try {
            userFeignClient.deleteUser(5);
        } catch (Exception e) {
            LOG.error(e.getMessage());
        }
        LOG.info("\n\n\nRetrieve users again\n\t\t\tto check if deleted user still exists");
        try {
            userFeignClient.getUser("y").forEach((UserVO user) -> {
                LOG.info("GET /v1/user --> {}", user);
            });
        } catch (Exception e) {
            LOG.error(e.getMessage());
        }
    }
}
```

```
|    }  
    }  
}
```

This is the way you can add other Feign client interfaces and consume them. You can explore more about it at <https://github.com/OpenFeign/feign>.

[Support](#) [Sign Out](#)

©2022 O'REILLY MEDIA, INC. [TERMS OF SERVICE](#) [PRIVACY POLICY](#)